1.Installing R and R Studio

Step 1-> Install R :

1. Go to the R Project website: https://cran.r-project.org
2. Select your operating system (Windows, macOS, or Linux).
3. Download R: For Windows, click on " Download R for Windows" -> " base" -> then click the .exe link.
4. Run the installer and follow the instructions (default settings are usually fine).

Step 2-> Install RStudio :

1. Go to the RStudio website: https://posit.co/download/rstudio-desktop
2. Choose the free RStudio Desktop (Open Source Edition).
3. Download the installer for your OS (Windows, macOS, or Linux).
4. Run the installer and follow the setup wizard.

Step 3-> Launch RStudio :

1. After installation, open RStudio from the Start menu or Desktop shortcut.
2. RStudio will automatically detect the installed R version.

Tips :

1. R is the language engine; RStudio is the IDE (Integrated Development Environment).
2. Always install R before RStudio.
3. After installation, you can write and run R programs using RStudio.

2.Applying simple commands in R

```
#create a numeric vector
numbers<-c(10,20,30,40,50)
#create a character vector
names<-c("alice","Bob","charlie","david","eva")
#combine into a data frame
data<-data.frame(name=names,score=numbers)
#view the dataframe
print(data)
#summary of data
summary(data)
#mean of the score column
mean_score<-mean(data$score)
print(mean_score)
#median of the score column
median_score<-median(data$score)
print(median_score)
#standard deviation
sd_score<-sd(data$score)
print(sd_score)
#add a new column
data$passed<-data$score>30
print(data)
#filter rows where passed is TRUE
passed_data<-subset(data,passed==TRUE)
print(passed_data)
#sort by score
sorted_data<-data[order(data$score),]
print(sorted_data)
#find max score
max_score<-max(data$score)
print(max_score)
#find min score
min_score<-min(data$score)
print(min_score)
#create a sequence
seq_nums<-seq(1,10,by=2)
print(seq_nums)
#repeat elements
rep_chars<-rep("R",times=5)
print(rep_chars)
#generate random numbers
rand_nums<-runif(5,min=1,max=100)
print(rand_nums)
name<-"machine learning"
version<-4.3
is_installed <-TRUE
#crating vectors
```

```
numbers<-c(1,2,3,4,5)
letters<-c("a","b","c","d")
#basic operations
sum_numbers<-sum(numbers)
mean_numbers<-mean(numbers)
max_numbers<-max(numbers)
min_numbers<-max(numbers)
#display results
print(paste("sum:",sum(numbers)))
print(paste("mean:",mean(numbers)))
print(paste("max:",max(numbers)))
#creating a data frame
student_data<-data.frame(
  name=c("alice","bob","charlie"),
  age=c(20,21,22),
  score=c(85,90,88)
)
print(student_data)
#sequence and repetition
seq_1_to_10<-seq(1,10,by=2)
rep_values<-rep(5,times=4)
print(seq(1, 10))
print(rep("values", 5))
```

Output :

```
#create a numeric vector
> numbers<-c(10,20,30,40,50)
> #create a character vector
> names<-c("alice","Bob","charlie","david","eva")
> #combine into a data frame
> data<-data.frame(name=names,score=numbers)
> #view the dataframe
> print(data)
     name score
1   alice    10
2     Bob    20
3 charlie    30
4   david    40
5     eva    50
> #summary of data
> summary(data)
     name               score
 Length:5           Min.   :10
 Class :character   1st Qu.:20
 Mode  :character   Median :30
                    Mean   :30
                    3rd Qu.:40
                    Max.   :50
```

```
> #mean of the score column
> mean_score<-mean(data$score)
> print(mean_score)
[1] 30
> #median of the score column
> median_score<-median(data$score)
> print(median_score)
[1] 30
> #standard deviation
> sd_score<-sd(data$score)
> print(sd_score)
[1] 15.81139
> #add a new column
> data$passed<-data$score>30
> print(data)
     name score passed
1   alice    10  FALSE
2     Bob    20  FALSE
3 charlie    30  FALSE
4   david    40   TRUE
5     eva    50   TRUE
> #filter rows where passed is TRUE
> passed_data<-subset(data,passed==TRUE)
> print(passed_data)
   name score passed
4 david    40   TRUE
5   eva    50   TRUE
> #sort by score
> sorted_data<-data[order(data$score),]
> print(sorted_data)
     name score passed
1   alice    10  FALSE
2     Bob    20  FALSE
3 charlie    30  FALSE
4   david    40   TRUE
5     eva    50   TRUE
> #find max score
> max_score<-max(data$score)
> print(max_score)
[1] 50
> #find min score
> min_score<-min(data$score)
> print(min_score)
[1] 10
> #create a sequence
> seq_nums<-seq(1,10,by=2)
> print(seq_nums)
[1] 1 3 5 7 9
> #repeat elements
> rep_chars<-rep("R",times=5)
> print(rep_chars)
```

```
[1] "R" "R" "R" "R" "R"
> #generate random numbers
> rand_nums<-runif(5,min=1,max=100)
> print(rand_nums)
[1] 25.48933 35.41771 73.10107 61.03680 91.94970
> name<-"machine learning"
> version<-4.3
> is_installed <-TRUE
> #crating vectors
> numbers<-c(1,2,3,4,5)
> letters<-c("a","b","c","d")
> #basic operations
> sum_numbers<-sum(numbers)
> mean_numbers<-mean(numbers)
> max_numbers<-max(numbers)
> min_numbers<-max(numbers)
> #display results
> print(paste("sum:",sum(numbers)))
[1] "sum: 15"
> print(paste("mean:",mean(numbers)))
[1] "mean: 3"
> print(paste("max:",max(numbers)))
[1] "max: 5"
> #creating a data frame
> student_data<-data.frame(
+   name=c("alice","bob","charlie"),
+   age=c(20,21,22),
+   score=c(85,90,88)
+ )
> print(student_data)
     name age score
1   alice  20    85
2     bob  21    90
3 charlie  22    88
> #sequence and repetition
> seq_1_to_10<-seq(1,10,by=2)
> rep_values<-rep(5,times=4)
```

3.R as a Calculator application

```
a<-15
b<-4
cat("addition:",a+b,"\n")
cat("subtraction:",a-b,"\n")
cat("multiplication:",a*b,"\n")
cat("division:",a/b,"\n")
cat("modulus:",a%%b,"\n")
cat("power:",a^b,"\n")

cat("\n--advanced functiomns--\n")
cat("square root of 144:",sqrt(144),"\n")
cat("absolute value of -25:",abs(-25),"\n")
cat("log of 100:",log(100),"\n")
cat("log 10 of 1000:",log10(1000),"\n")
cat("exponential e^2:",exp(2),"\n")

cat("\n--trigonometric functions--\n")
angle<-pi/4#45 degrees
cat("sin(π/4):",sin(angle),"\n")
cat("cos(π/4):",cos(angle),"\n")
cat("tan(π/4):",tan(angle),"\n")

data<-c(12,15,18,22,25,30,35)
cat("\n--statistical calculations--\n")
cat("data:",data,"\n")
cat("mean:",mean(data),"\n")
cat("median:",median(data),"\n")
cat("standard deviation:",sd(data),"\n")
cat("variance:",var(data),"\n")
cat("range:",range(data),"\n")
```

Output :

```
> a<-15
> b<-4
> cat("addition:",a+b,"\n")
addition: 19
> cat("subtraction:",a-b,"\n")
subtraction: 11
> cat("multiplication:",a*b,"\n")
multiplication: 60
> cat("division:",a/b,"\n")
```

```
division: 3.75
> cat("modulus:",a%%b,"\n")
modulus: 3
> cat("power:",a^b,"\n")
power: 50625
>
> cat("\n--advanced functiomns--\n")

--advanced functiomns--
> cat("square root of 144:",sqrt(144),"\n")
square root of 144: 12
> cat("absolute value of -25:",abs(-25),"\n")
absolute value of -25: 25
> cat("log of 100:",log(100),"\n")
log of 100: 4.60517
> cat("log 10 of 1000:",log10(1000),"\n")
log 10 of 1000: 3
> cat("exponential e^2:",exp(2),"\n")
exponential e^2: 7.389056
>
> cat("\n--trigonometric functions--\n")

--trigonometric functions--
> angle<-pi/4#45 degrees
> cat("sin(π/4):",sin(angle),"\n")
sin(π/4): 0.7071068
> cat("cos(π/4):",cos(angle),"\n")
cos(π/4): 0.7071068
> cat("tan(π/4):",tan(angle),"\n")
tan(π/4): 1
>
> data<-c(12,15,18,22,25,30,35)
> cat("\n--statistical calculations--\n")

--statistical calculations--
> cat("data:",data,"\n")
data: 12 15 18 22 25 30 35
> cat("mean:",mean(data),"\n")
mean: 22.42857
> cat("median:",median(data),"\n")
median: 22
> cat("standard deviation:",sd(data),"\n")
standard deviation: 8.22308
> cat("variance:",var(data),"\n")
variance: 67.61905
> cat("range:",range(data),"\n")
range: 12 35
```

4.Execution of Loops via R-control structures

```
cat("--IF ELSE Example--\n")
score<-85
if(score>=90){
  grade<-"a"
}else if(score>=80){
  grade<-"b"
}else if(score>=70){
  grade<-"c"
}else{
  grade<-"f"
}
cat("score:",score,"grade",grade,"\n")

cat("\n--for loop examples--\n")
cat("counting 1 to 5:")
for(i in 1:5){
  cat(i," ")
}

fruits<-c("apple","banana","cherry")
cat("\n\nfruits:\n")
for(fruit in fruits){
  cat("-",fruit,"\n")
}

cat("multiplication table(3x3):\n")
for(i in 1:3){
  for(j in 1:3){
    cat(i*j,"\t")
  }
  cat("\n")
}

cat("\n--whileloop--\n")
count<-1
while(count<=5){
  cat("count:",count,"\n")
  count<-count+1
}

cat("\n--repeat loop--\n")
num<-1
repeat{
  cat("number:",num,"\n")
  num<-num+1
  if(num>5){
    break
```

```
  }
}

cat("\n--using next statement--\n")
for(i in i:10){
  if(i%%2==0){
    next#skip even numbers
  }
  cat("odd numbers:","\n")
}
```

Output :

```
> cat("--IF ELSE Example--\n")
--IF ELSE Example--
> score<-85
> if(score>=90){
+   grade<-"a"
+ }else if(score>=80){
+   grade<-"b"
+ }else if(score>=70){
+   grade<-"c"
+ }else{
+   grade<-"f"
+ }
> cat("score:",score,"grade",grade,"\n")
score: 85 grade b
>
> cat("\n--for loop examples--\n")

--for loop examples--
> cat("counting 1 to 5:")
counting 1 to 5:> for(i in 1:5){
+   cat(i," ")
+ }
1  2  3  4  5  >
> fruits<-c("apple","banana","cherry")
> cat("\n\nfruits:\n")


fruits:
> for(fruit in fruits){
+   cat("-",fruit,"\n")
+ }
- apple
- banana
- cherry
>
```

```
> cat("multiplication table(3x3):\n")
multiplication table(3x3):
> for(i in 1:3){
+   for(j in 1:3){
+     cat(i*j,"\t")
+   }
+   cat("\n")
+ }
1       2       3
2       4       6
3       6       9
>
> cat("\n--whileloop--\n")

--whileloop--
> count<-1
> while(count<=5){
+   cat("count:",count,"\n")
+   count<-count+1
+ }
count: 1
count: 2
count: 3
count: 4
count: 5
>
> cat("\n--repeat loop--\n")

--repeat loop--
> num<-1
> repeat{
+   cat("number:",num,"\n")
+   num<-num+1
+   if(num>5){
+     break
+   }
+ }
number: 1
number: 2
number: 3
number: 4
```

5.Execution of functions using R

```
cat("----simple function----\n")
greet<-function(name){
  return(paste("hello",name,"welcome R programming"))
}

print(greet("students"))
cat("\n--function with multiple parameters--\n")
calculate_area<-function(length,width){
  area<-length*width
  return(area)
}

cat("area of rectangle(5x3):",calculate_area(5,3),"\n")
cat("area of rectangle(10x7):",calculate_area(10,7),"\n")
cat("\n--- function with default parameters--\n")
power_calc<-function(base,exponent=2){
  return(base^exponent)
}

cat("5^2=",power_calc(5),"\n")
cat("5^3=",power_calc(5,3),"\n")
cat("2^4=",power_calc(2,4),"\n")
cat("\n--- function returning multiple values--\n")
stats_summary<-function(data){
  result<-list(
    mean=mean(data),
    median=median(data),
    sd=sd(data),
    min=min(data),
    max=max(data)
)

return(result)
}
numbers<-c(10,20,30,40,50)
summary_result<-stats_summary(numbers)
print(summary_result)

cat("\n---recursive function(factorial)--\n")
factorial<-function(n){
  if(n<=1){
    return(1)
  }else{
    return(n*factorial(n-1))
  }
}
```

```
cat("Factorial of 5:",factorial(5),"\n")
cat("Factorial of 7:",factorial(7),"\n")
cat("\n---function with variable arguments--\n")
sum_all<-function(...){
  args<-list(...)
  return(sum(unlist(args)))
}

cat("sum of 1,2,3,4,5:",sum_all(1,2,3,4,5),"\n")
cat("sum of 10,20,30:",sum_all(10,20,30),"\n")
cat("\n--anonymous function--\n")
numbers_vec<-c(1,2,3,4,5)
squared<-sapply(numbers_vec,function(x)x^2)
cat("squared values:",squared,"\n")
```

Output :

```
> cat("----simple function----\n")
----simple function----
> greet<-function(name){
+   return(paste("hello",name,"welcome R programming"))
+ }
>
> print(greet("students"))
[1] "hello students welcome R programming"
> cat("\n--function with multiple parameters--\n")

--function with multiple parameters--
> calculate_area<-function(length,width){
+   area<-length*width
+   return(area)
+ }
>
> cat("area of rectangle(5x3):",calculate_area(5,3),"\n")
area of rectangle(5x3): 15
> cat("area of rectangle(10x7):",calculate_area(10,7),"\n")
area of rectangle(10x7): 70
> cat("\n--- function with default parameters--\n")

--- function with default parameters--
> power_calc<-function(base,exponent=2){
+   return(base^exponent)
+ }
>
> cat("5^2=",power_calc(5),"\n")
5^2= 25
> cat("5^3=",power_calc(5,3),"\n")
```

```
5^3= 125
> cat("2^4=",power_calc(2,4),"\n")
2^4= 16
> cat("\n--- function returning multiple values--\n")

--- function returning multiple values--
> stats_summary<-function(data){
+   result<-list(
+     mean=mean(data),
+     median=median(data),
+     sd=sd(data),
+     min=min(data),
+     max=max(data)
+   )
+
+   return(result)
+ }
> numbers<-c(10,20,30,40,50)
> summary_result<-stats_summary(numbers)
> print(summary_result)
$mean
[1] 30

$median
[1] 30

$sd
[1] 15.81139

$min
[1] 10

$max
[1] 50

>
> cat("\n---recursive function(factorial)--\n")

---recursive function(factorial)--
> factorial<-function(n){
+   if(n<=1){
+     return(1)
+   }else{
+     return(n*factorial(n-1))
+   }
+ }
>
> cat("Factorial of 5:",factorial(5),"\n")
Factorial of 5: 120
> cat("Factorial of 7:",factorial(7),"\n")
Factorial of 7: 5040
```

```
> cat("\n---function with variable arguments--\n")

---function with variable arguments--
> sum_all<-function(...){
+   args<-list(...)
+   return(sum(unlist(args)))
+ }
>
> cat("sum of 1,2,3,4,5:",sum_all(1,2,3,4,5),"\n")
sum of 1,2,3,4,5: 15
> cat("sum of 10,20,30:",sum_all(10,20,30),"\n")
sum of 10,20,30: 60
> cat("\n--anonymous function--\n")

--anonymous function--
> numbers_vec<-c(1,2,3,4,5)
> squared<-sapply(numbers_vec,function(x)x^2)
> cat("squared values:",squared,"\n")
squared values: 1 4 9 16 25
>
```

6.Reading and writing different types of datasets in R

```
# Create sample data
cat("--- Creating Sample Dataset ---\n")
student_data <- data.frame(
  StudentID = c(101, 102, 103, 104, 105),
  Name = c("Alice", "Bob", "Charlie", "Diana", "Edward"),
  Age = c(20, 21, 22, 20, 23),
  Grade = c("A", "B+", "A-", "B", "A+"),
  Score = c(85, 78, 88, 72, 92),
  stringsAsFactors = FALSE
)
print(student_data)
# Writing to CSV
cat("\n--- Writing to CSV File ---\n")
write.csv(student_data, "students.csv", row.names = FALSE)
cat("Data written to students.csv\n")
# Reading from CSV
cat("\n--- Reading from CSV File ---\n")
read_data <- read.csv("students.csv")
print(read_data)
# Writing to TXT
cat("\n--- Writing to TXT File ---\n")
write.table(student_data, "students.txt",
            row.names = FALSE, sep = "\t")
cat("Data written to students.txt\n")
# Reading from TXT
cat("\n--- Reading from TXT File ---\n")
txt_data <- read.table("students.txt", header = TRUE)
print(txt_data)
# Working with RData format
cat("\n--- Working with RData Format ---\n")
save(student_data, file = "students.RData")
cat("Data saved as students.RData\n")

# Reading specific columns
cat("\n--- Reading Specific Columns ---\n")
csv_data <- read.csv("students.csv",
                     colClasses = c("integer", "character",
                                    "integer", "character",
"integer"))
names_only <- csv_data[, c("Name", "Score")]
print(names_only)

# Reading with conditions
cat("\n--- Reading with Conditions ---\n")
high_scorers <- subset(csv_data, Score >= 80)
print("Students with Score >= 80:")
print(high_scorers)
```

```
# Summary statistics
cat("\n--- Summary Statistics ---\n")
summary(csv_data)
```

Output :

```
> cat("\n--- Reading from TXT File ---\n")

--- Reading from TXT File ---
> txt_data <- read.table("students.txt", header = TRUE)
> print(txt_data)
  StudentID    Name Age Grade Score
1       101   Alice  20     A    85
2       102     Bob  21    B+    78
3       103 Charlie  22    A-    88
4       104   Diana  20     B    72
5       105  Edward  23    A+    92
> # Working with RData format
> cat("\n--- Working with RData Format ---\n")

--- Working with RData Format ---
> save(student_data, file = "students.RData")
> cat("Data saved as students.RData\n")
Data saved as students.RData
>
> # Reading specific columns
> cat("\n--- Reading Specific Columns ---\n")

--- Reading Specific Columns ---
> csv_data <- read.csv("students.csv",
+                      colClasses = c("integer", "character",
+                                     "integer", "character",
"integer"))
> names_only <- csv_data[, c("Name", "Score")]
> print(names_only)
     Name Score
1   Alice    85
2     Bob    78
3 Charlie    88
4   Diana    72
5  Edward    92
>
> # Reading with conditions
> cat("\n--- Reading with Conditions ---\n")

--- Reading with Conditions ---
> high_scorers <- subset(csv_data, Score >= 80)
```

```
> print("Students with Score >= 80:")
[1] "Students with Score >= 80:"
> print(high_scorers)
  StudentID    Name Age Grade Score
1       101   Alice  20     A    85
3       103 Charlie  22    A-    88
5       105  Edward  23    A+    92
>
> # Summary statistics
> cat("\n--- Summary Statistics ---\n")

--- Summary Statistics ---
> summary(csv_data)
   StudentID       Name                Age           Grade
 Min.   :101   Length:5           Min.   :20.0   Length:5
 1st Qu.:102   Class :character   1st Qu.:20.0   Class
:character
 Median :103   Mode  :character   Median :21.0   Mode
:character
 Mean   :103                      Mean   :21.2
 3rd Qu.:104                      3rd Qu.:22.0
 Max.   :105                      Max.   :23.0
     Score
 Min.   :72
 1st Qu.:78
 Median :85
 Mean   :83
 3rd Qu.:88
 Max.   :92
>
```

7.Visualize various plotting and graphics in R

```
#Dataset
students <- data.frame(
  Name = c("Asha", "Bala", "Charan", "Divya", "Fathima",
           "Gokul", "Hari", "Irfan", "Jaya", "Kavin"),
  Marks = c(85, 72, 90, 68, 78, 95, 88, 76, 82, 91),
  Attendance = c(92, 85, 96, 78, 88, 98, 94, 82, 90, 97)
)
print(students)
# 2. Bar Plot
barplot(
  students$Marks,
  names.arg = students$Name,
  main = "Student Marks",
  xlab = "Students",
  ylab = "Marks",
  col = "skyblue",
  border = "black"
)
# 3. Pie Chart
pie(
  students$Marks,
  labels = students$Name,
  main = "Distribution of Student Marks",
  col = rainbow(length(students$Marks))
)


# 4. Histogram
hist(
  students$Marks,
  main = "Histogram of Student Marks",
  xlab = "Marks",
  ylab = "Frequency",
  col = "lightgreen",
  border = "black"
)
# 5. Box Plot
boxplot(
  students$Marks,
  main = "Box Plot of Student Marks",
  ylab = "Marks",
```

```
  col = "orange"
)


# 6. Scatter Plot
plot(
  students$Attendance,
  students$Marks,
  main = "Attendance vs Marks",
  xlab = "Attendance (%)",
  ylab = "Marks",
  pch = 19,
  col = "blue"
)

# Add regression line
abline(
  lm(Marks ~ Attendance, data = students),
  col = "red",
  lwd = 2
)
# 7. Line Plot
plot(
  students$Marks,
  type = "o",
  main = "Student Marks Trend",
  xlab = "Student Number",
  ylab = "Marks",
  pch = 16,
  lwd = 2,
  col = "darkgreen"
)

# 8. Density Plot
density_marks <- density(students$Marks)
plot(
  density_marks,
  main = "Density Plot of Student Marks",
  xlab = "Marks",
  ylab = "Density",
  lwd = 2,
  col = "purple"
)
```

```
# 9. Multiple Plots in One Window

par(mfrow = c(2, 2))

# Bar Plot
barplot(
  students$Marks,
  names.arg = students$Name,
  main = "Bar Plot",
  col = "skyblue",
  las = 2
)

# Histogram
hist(
  students$Marks,
  main = "Histogram",
  col = "lightgreen"
)

# Box Plot
boxplot(
  students$Marks,
  main = "Box Plot",
  col = "orange"
)

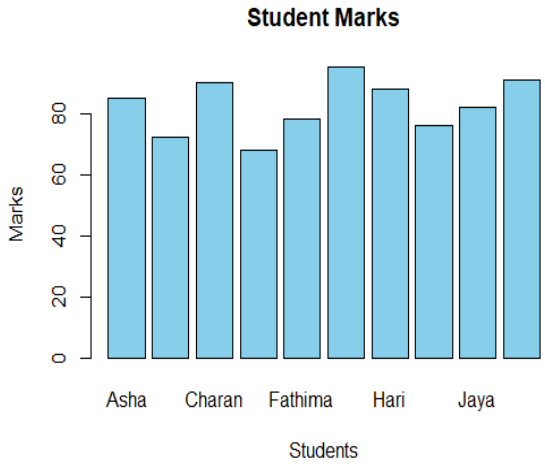

# Scatter Plot
plot(
  students$Attendance,
  students$Marks,
  main = "Scatter Plot",
  xlab = "Attendance",
  ylab = "Marks",
  pch = 19,
  col = "blue"
)

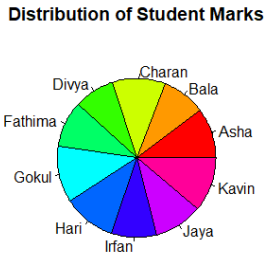
# Reset plotting layout
par(mfrow = c(1, 1))
# 10. Summary Statistics
cat("Mean Marks:", mean(students$Marks), "\n")
```

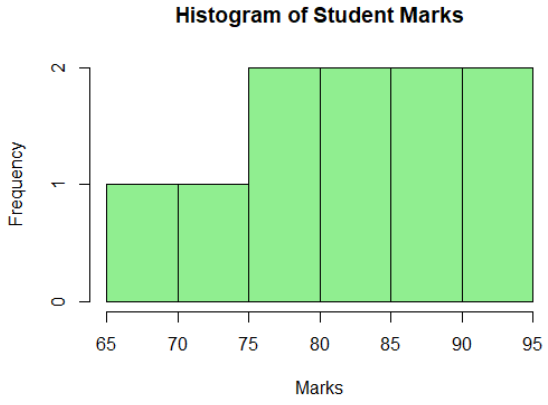
```
cat("Median Marks:", median(students$Marks), "\n")
cat("Maximum Marks:", max(students$Marks), "\n")
cat("Minimum Marks:", min(students$Marks), "\n")
cat("Standard Deviation:", sd(students$Marks), "\n")
```

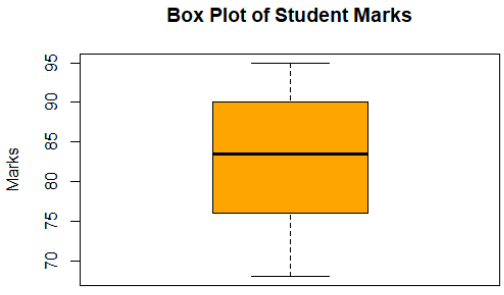
Output :

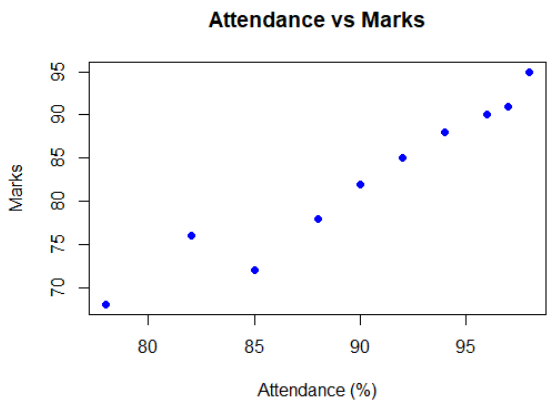
```
#Dataset
students <- data.frame(
  Name = c("Asha", "Bala", "Charan", "Divya", "Fathima",
           "Gokul", "Hari", "Irfan", "Jaya", "Kavin"),
  Marks = c(85, 72, 90, 68, 78, 95, 88, 76, 82, 91),
  Attendance = c(92, 85, 96, 78, 88, 98, 94, 82, 90, 97)
)
print(students)
```

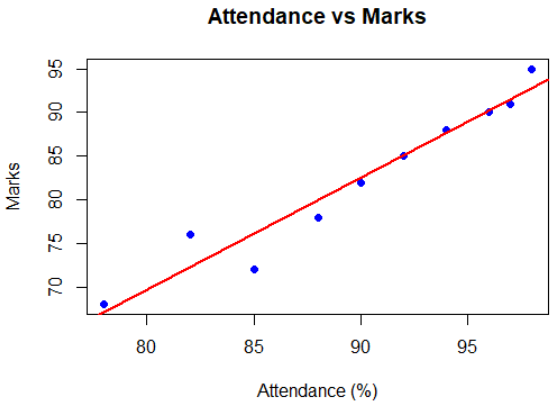


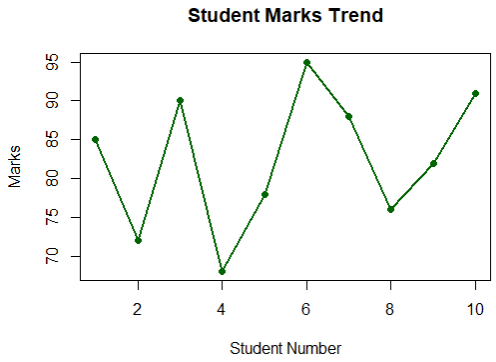


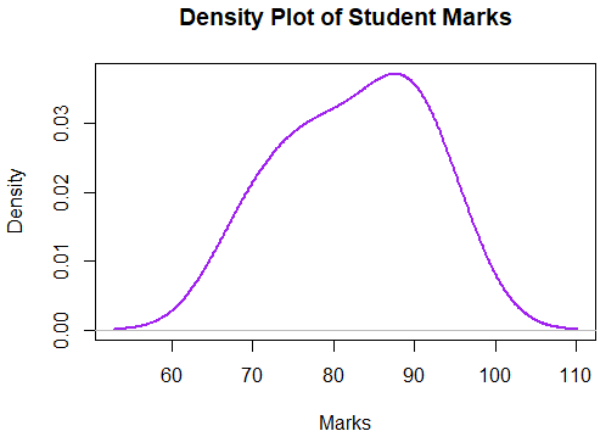





Attendance vs Marks
Marks
Attendance (%)
70
75
80
85
90
95
80
85
90
95


Attendance vs Marks
Marks
Attendance (%)
70
75
80
85
90
95
80
85
90
95


Student Marks Trend
Marks
Student Number
70
75
80
85
90
95
2
4
6
8
10


Density Plot of Student Marks
Density
Marks
0.00
0.01
0.02
0.03
60
70
80
90
100
110

8.Perform simple regression using R package

```
# Simple Linear Regression in R

# Create data

x <- c(1, 2, 3, 4, 5)

y <- c(2, 4, 5, 4, 6)

# Create a linear regression model

model <- lm(y ~ x)

# Display the model result

summary(model)

# Predict values

predicted <- predict(model)

# Display predicted values

print(predicted)

# Plot the data and regression line

plot(x, y,

    main = "Simple Linear Regression",

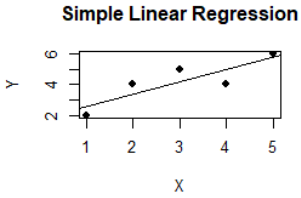
    xlab = "X",

    ylab = "Y",

    pch = 19)

abline(model)
```

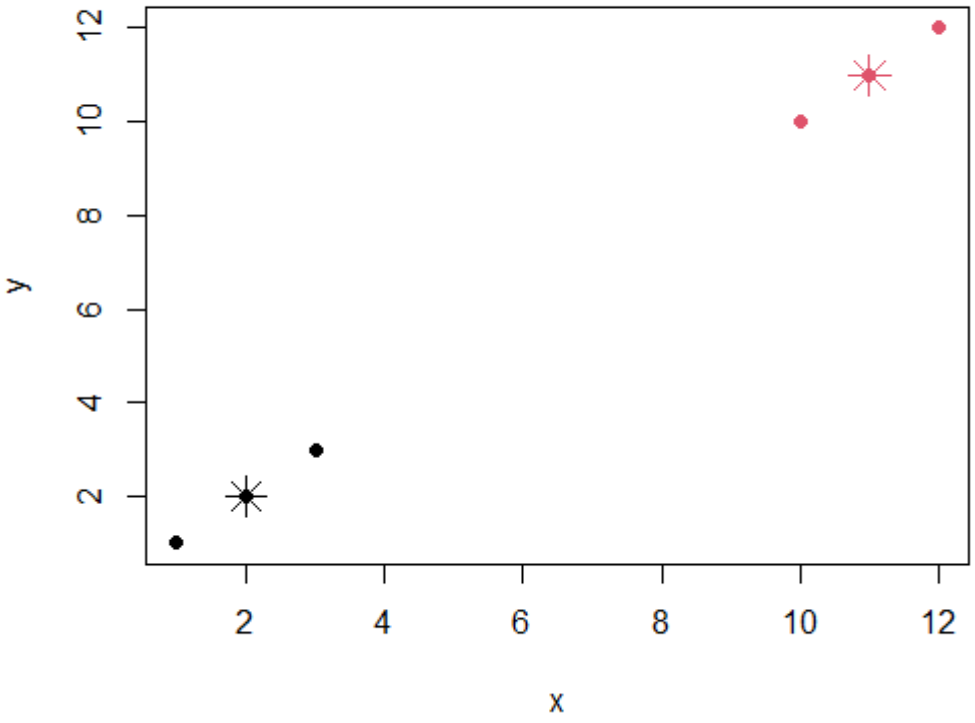
Output :


Simple Linear Regression
Y
2
4
6
1
2
3
4
5
X

9. Apply k-means by using R package

```
# K-Means Clustering
data <- data.frame(
  x = c(1,2,3,10,11,12),
  y = c(1,2,3,10,11,12)
)
result <- kmeans(data, centers=2)
print(result$cluster)
print(result$centers)
plot(data, col=result$cluster, pch=19)
points(result$centers, col=1:2, pch=8, cex=2)
```

Output:



10. Use decision Tree/Random Forest/Naïve Bayes / NN by using R package

```
# Decision Tree
install.packages("rpart")
library(rpart)
data(iris)
model <- rpart(Species ~ ., data=iris, method="class")
print(model)
plot(model)
text(mode)
# Random Forest
install.packages("randomForest")
library(randomForest)
data(iris)
model <- randomForest(Species ~ ., data=iris)
print(model)
predict(model, iris[1:5,])

# Naive Bayes
install.packages("e1071")
library(e1071)
data(iris)
model <- naiveBayes(Species ~ ., data=iris)
print(model)
predict(model, iris[1:5,])
```

```
# Neural Network

install.packages("neuralnet")

library(neuralnet)

data(iris)

iris$Y <- ifelse(iris$Species=="setosa",1,0)

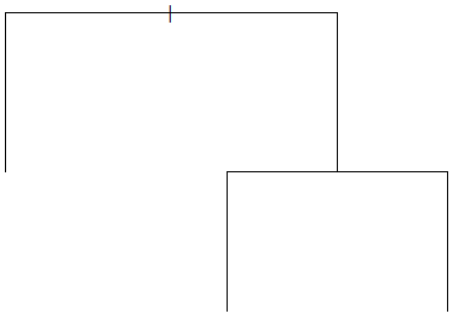
model <- neuralnet(Y ~ Sepal.Length + Sepal.Width,

            data=iris)

print(model)

plot(model)
```

Output:



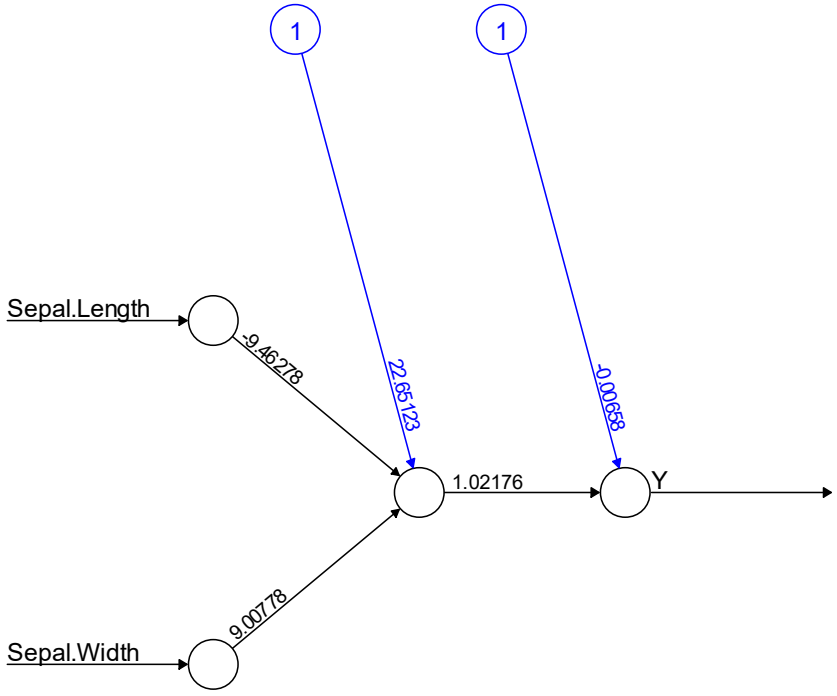
```
Call:
 randomForest(formula = Species ~ ., data = iris)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 5.33%
Confusion matrix:
           setosa versicolor virginica class.error
setosa         50          0         0        0.00
versicolor      0         47         3        0.06
virginica       0          5        45        0.10
>
> predict(model, iris[1:5,])
     1      2      3      4      5
```

```
setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica


 package ‘randomForest’ successfully unpacked and MD5 sums checked

 The downloaded binary packages are in
        C:\Users\STUDENT\AppData\Local\Temp\Rtmpmm4hEM\downloaded_packages
```



Error: 0.123782 Steps: 5818